

Assignment 2

Part A: The Manual Implementation (Proof of Concept)

1. **Generate a Signal:** Create a 1D dataset (e.g., a simple sine wave with added noise).
2. **The Calculus Step:** Write a Python function `manual_dft(signal)` that computes the transform **without** using `numpy.fft`.
 - You must use a `for` loop.
 - This demonstrates your understanding of the Riemann Sum approximation.
3. **Compare:** Compare your manual calculation against the optimized library version to verify accuracy.

Part B: 2D Image Processing (Application)

1. **Load a Grayscale Image:** Treat the image as a 2D function $f(x, y)$.
2. **Transform:** Apply the Fourier Transform to move from the *Spatial Domain* to the *Frequency Domain*.
3. **Filter (Integration vs. Differentiation):**
 - **Low-Pass Filter (Smoothing):** Remove high frequencies. Relate this to **Integration** (which smooths functions/accumulates area).
 - **High-Pass Filter (Edge Detection):** Remove low frequencies. Relate this to **Differentiation** (which highlights rates of change/edges).
4. **Inverse Transform:** Convert the filtered frequencies back into an image using the Inverse Fourier Transform (reconstructing the function from its series).

4. Deliverables

1. **Python Code:** A `.py` file containing your manual DFT implementation and the image processing pipeline.

5. Rubric

- **30%** - Correct implementation of the manual Riemann Sum loop.
- **30%** - Successful Low-Pass and High-Pass filtering of an image.
- **20%** - Visualizations (Original vs. Spectrum vs. Filtered).
- **20%** - Explanation of the link between the Code and the Calculus Integral.